

გაკვეთილი 1 Mod 7 (7.1)

გაკვეთილი 1 — ვერსიების კონტროლი და Git-ის საფუძვლები

1. გაკვეთილის დასაწყისი — პრობლემის ჩვენება

გაკვეთილი შეგვიძლია დავიწყოთ მარტივი სიტუაციით.

ვთქვათ, ვაკეთებთ ვებგვერდს და გვაქვს ფაილები:

```
index.html  
style.css  
script.js
```

დღეს საიტი კარგად მუშაობს. ხვალ `style.css`-ში ბევრი რამ შევცვალეთ და დიზაინი გაფუჭდა.

გვინდა ძველ მდგომარეობაზე დაბრუნება.

როგორ მოვიქცეთ?

ერთ-ერთი მარტივი გზა იქნებოდა ფაილების ასლების შენახვა:

```
project  
project-final  
project-final2  
project-final-real  
project-final-real2  
project-last
```

პატარა პროექტში შეიძლება ეს მეთოდი გამოვიყენოთ, მაგრამ დიდ პროექტში მალე ვეღარ გავიგებთ:

- რომელი ვერსიაა ბოლო?
- რა შევცვალეთ?
- როდის შევცვალეთ?
- ვინ შეცვალა?
- როგორ დავაბრუნოთ ძველი ვერსია?

სწორედ ამ პრობლემის გადასაჭრელად არსებობს **Version Control System** — ვერსიების კონტროლის სისტემა.

2. რა არის ვერსიების კონტროლი?

ვერსიების კონტროლი არის სისტემა, რომელიც პროექტში განხორციელებულ ცვლილებებს ინახავს და გვაძლევს მათი ისტორიის მართვის საშუალებას.

მარტივად:

ვერსიების კონტროლი ინახავს ჩვენი პროექტის ცვლილებების ისტორიას.

მაგალითად, ვებგვერდზე ვმუშაობთ.

თავიდან გვაქვს:

```
<h1>My Website</h1>
```

შემდეგ დავამატეთ:

```
<h1>My Website</h1>
<p>Welcome to my website!</p>
```

შემდეგ დავამატეთ ღილაკი:

```
<h1>My Website</h1>
<p>Welcome to my website!</p>
<button>Contact Me</button>
```

ვერსიების კონტროლის გამოყენების შემთხვევაში შეგვიძლია გვქონდეს ასეთი ისტორია:

```
Version 1 → შექმნილია მთავარი გვერდი
Version 2 → დამატებულია ტექსტი
Version 3 → დამატებულია ღილაკი
```

თუ მესამე ვერსიაში პრობლემა წარმოიშვა, შეგვიძლია ვნახოთ წინა ცვლილებები და საჭიროების შემთხვევაში ძველ მდგომარეობას დავუბრუნდეთ.

3. რატომ გვჭირდება ვერსიების კონტროლი?

ვერსიების კონტროლი განსაკუთრებით მნიშვნელოვანია პროგრამირებაში.

მისი საშუალებით შეგვიძლია:

ცვლილებების ისტორიის შენახვა — ვიცით, რა შეიცვალა პროექტში.

ძველ ვერსიაზე დაბრუნება — თუ კოდი გავაფუჭეთ, შესაძლებელია წინა მდგომარეობის აღდგენა.

ცვლილებების შედარება — შეგვიძლია ვნახოთ, კონკრეტულად რა დავამატეთ ან წავშალეთ.

გუნდურად მუშაობა — რამდენიმე პროგრამისტს შეუძლია ერთ პროექტზე მუშაობა.

ექსპერიმენტების უსაფრთხოდ ჩატარება — შეგვიძლია ახალი ფუნქცია ცალკე ვცადოთ ისე, რომ ძირითადი პროექტი არ გავაფუჭოთ.

4. რა არის Git?

Git არის ვერსიების კონტროლის სისტემა.

მისი მთავარი დანიშნულებაა პროექტში განხორციელებული ცვლილებების კონტროლი და ისტორიის შენახვა.

Git კომპიუტერზე მუშაობს.

მაგალითად, გვაქვს:

```
my-website/  
  index.html  
  style.css  
  script.js
```

Git-ს შეგვიძლია ვუთხრათ:

„ამ პროექტში განხორციელებული ცვლილებების კონტროლი დაიწყე.“

ამის შემდეგ Git შეძლებს დააკვირდეს პროექტის ფაილებში განხორციელებულ ცვლილებებს.

მნიშვნელოვანია:

Git ≠ GitHub

ეს ორი სხვადასხვა რამეა.

5. Git და GitHub — რა განსხვავებაა?

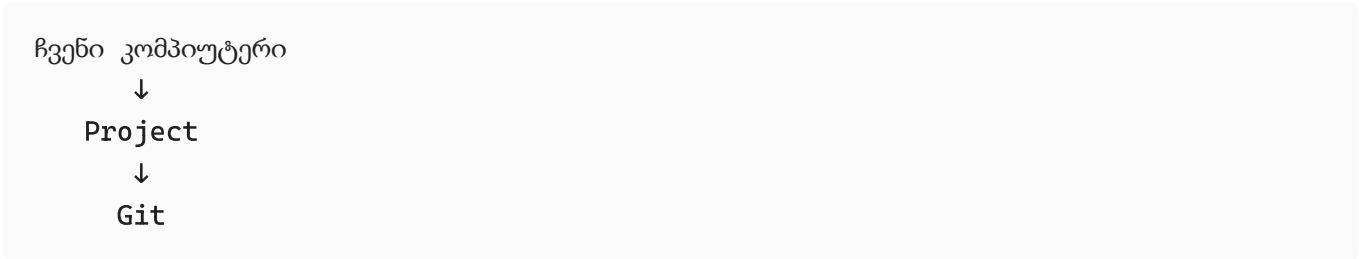
ეს განსხვავება თავიდანვე კარგად უნდა ვიცოდეთ.

Git

Git არის ვერსიების კონტროლის სისტემა.

იგი ჩვენს კომპიუტერზე მუშაობს და პროექტის ცვლილებების მართვაში გვეხმარება.

მაგალითად:



Git-ის გამოყენებისთვის ინტერნეტი ყოველთვის საჭირო არ არის.

GitHub

[GitHub](#) არის ონლაინ პლატფორმა, სადაც შეგვიძლია Git-ით მართული პროექტები ინტერნეტში შევინახოთ და სხვებსაც გავუზიაროთ.

მარტივი შედარება:

Git	GitHub
ვერსიების კონტროლის სისტემა	ონლაინ პლატფორმა
მუშაობს ჩვენს კომპიუტერზე	მუშაობს ინტერნეტში
აკონტროლებს ცვლილებებს	ინახავს და აზიარებს Git პროექტებს
ინტერნეტი აუცილებელი არაა	ინტერნეტი საჭიროა

შეგვიძლია ასე დავიმახსოვროთ:

Git მართავს პროექტის ვერსიებს, **GitHub** კი საშუალებას გვაძლევს **Git** პროექტი ონლაინ შევინახოთ და გავაზიაროთ.

GitHub-ს უფრო დეტალურად მესამე გაკვეთილზე გამოვიყენებთ.

6. რა არის Repository?

Git-ში ერთ-ერთი ყველაზე მნიშვნელოვანი სიტყვაა:

Repository ან მოკლედ **Repo**.

Repository არის პროექტი/საქაღალდე, რომლის ცვლილებებსაც Git აკონტროლებს.

მაგალითად გვაქვს:

```
school-website/  
|  
├─ index.html  
├─ about.html  
├─ style.css  
└─ script.js
```

თუ ამ საქაღალდეში Git-ის ვერსიების კონტროლს ჩავრთავთ, ის გახდება **Git Repository**.

ანუ:

```
ჩვეულებრივი საქაღალდე  
↓  
git init  
↓  
Git Repository
```

7. Git-ის დაყენების შემოწმება

გავხსნათ:

Windows: Terminal / Command Prompt / Git Bash

და ჩავწეროთ:

```
git --version
```

თუ Git დაყენებულია, მივიღებთ დაახლოებით ასეთ პასუხს:

```
git version 2.51.0
```

ზუსტი ნომერი შეიძლება განსხვავდებოდეს.

თუ ბრძანება ვერ მოიძებნა, საჭიროა Git-ის დაყენება [Git-ის ოფიციალური საიტიდან](#).

8. პირველი Git Repository-ის შექმნა

ახლა პრაქტიკულად შევქმნათ პირველი repository.

შევქმნათ საქაღალდე:

```
my-first-git-project
```

შიგნით შევვიძლია შევქმნათ:

```
index.html
```

მაგალითად:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Git Project</title>
</head>
<body>
  <h1>Hello Git!</h1>
</body>
</html>
```

შემდეგ Terminal-ით უნდა შევიდეთ ამ საქაღალდეში.

მაგალითად:

```
cd my-first-git-project
```

9. git init

ახლა ვწერთ:

```
git init
```

`init` მოდის სიტყვიდან **initialize** — ინიციალიზაცია/დაწყება.

ამ ბრძანებით Git-ს ვეუბნებით:

„ამ საქალაქში დაიწყე ვერსიების კონტროლი.“

შედგად დაახლოებით ასეთ შეტყობინებას მივიღებთ:

```
Initialized empty Git repository...
```

ამ მომენტიდან ჩვენი საქალაქე უკვე **Git Repository**-ია.

მნიშვნელოვანია, რომ:

```
git init
```

პროექტის ფაილებს არ ცვლის და არ შლის.

იგი ქმნის დამალულ საქალაქედს:

```
.git
```

სწორედ `.git` -ში ინახავს Git პროექტის შესახებ ინფორმაციასა და ცვლილებების ისტორიას.

`.git` საქალაქდის ხელით შეცვლა ან წაშლა არ არის საჭირო.

10. `git status`

შემდეგი მნიშვნელოვანი ბრძანებაა:

```
git status
```

ეს ბრძანება გვაჩვენებს ჩვენი Git Repository-ის მიმდინარე მდგომარეობას.

მარტივად:

`git status` → „Git, მითხარი ახლა რა მდგომარეობაშია ჩემი პროექტი.“

თუ გვაქვს `index.html` , Git შეიძლება გვაჩვენოს:

Untracked files:

`index.html`

აქ ახალი სიტყვაა:

Untracked

Untracked file არის ფაილი, რომლის არსებობაც Git-მა დაინახა, მაგრამ ჯერ არ გვითქვამს, რომ მისი ცვლილებების ისტორიის შენახვა გვინდა.

ანუ:

`index.html`

↓

Git ხედავს ფაილს

↓

მაგრამ ჯერ არ აკონტროლებს

↓

UNTRACKED

მის კონტროლს შემდეგ გაკვეთილზე `git add` -ით დავიწყებთ.

11. Git-ის მუშაობის ძირითადი იდეა

ახლა მოსწავლეებს შეგვიძლია წინასწარ ვაჩვენოთ სრული პროცესი:

ვცვლით კოდს

↓

`git status`

↓

`git add`

↓

`git commit`

↓

ცვლილება ინახება ისტორიაში

ამ გაკვეთილზე ძირითადი ყურადღება გვაქვს:

```
Project
  ↓
git init
  ↓
Git Repository
  ↓
git status
```

`git add` და `git commit` შემდეგ გაკვეთილზე დეტალურად იქნება.

12. Working Directory, Staging Area და Repository

პირველ გაკვეთილზე ეს ცნებები შეგვიძლია მარტივად გავაცნოთ, რადგან შემდეგ გაკვეთილზე დაგვჭირდება.

Git-ში მუშაობისას ძირითადად გვაქვს სამი ეტაპი:

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Repository
```

1. Working Directory

ეს არის პროექტი, რომელშიც ახლა ვმუშაობთ.

მაგალითად:

```
index.html
style.css
script.js
```

როდესაც `index.html`-ს ვცვლით, ცვლილება ჯერ **Working Directory**-შია.

2. Staging Area

აქ ვირჩევთ, რომელი ცვლილებების შენახვა გვინდა შემდეგ ვერსიაში.

ამისთვის მოგვიანებით გამოვიყენებთ:

```
git add
```

3. Repository

აქ უკვე ინახება პროექტის შენახული ვერსიების ისტორია.

ამისთვის გამოვიყენებთ:

```
git commit
```

მარტივი ანალოგია:

Working Directory — ვმუშაობ დავალებაზე.

Staging Area — ავირჩიე, რისი ჩაბარება მინდა.

Repository/Commit — დავალების ეს ვერსია უკვე შევინახე.

13. პრაქტიკული მაგალითი კლასში

მოსწავლეებმა შექმნან:

```
git-practice
```

მასში შექმნან:

```
index.html
```

და ჩაწერონ:

```
<!DOCTYPE html>
<html>
<head>
  <title>Git Practice</title>
</head>
<body>
  <h1>My First Git Project</h1>
  <p>I am learning Git.</p>
</body>
</html>
```

Terminal-ში შევიდნენ პროექტის საქაღალდეში და შეასრულონ:

```
git init
```

შემდეგ:

```
git status
```

უნდა დაინახონ, რომ `index.html` არის **untracked file**.

ახლა `index.html` -ში დაამატონ:

```
<button>Click Me</button>
```

და ისევ გაუშვან:

```
git status
```

აქ მთავარი მიზანი ჯერ ცვლილების შენახვა კი არაა, არამედ გაიგონ, რომ **Git** პროექტის მდგომარეობას აკვირდება და `git status` ამ მდგომარეობის სანახავად გამოიყენება.

14. სავარჯიშო 1 — თეორიული

შეავსონ გამოტოვებული ადგილები:

1. ___ არის ვერსიების კონტროლის სისტემა.
2. ___ არის ონლაინ პლატფორმა, სადაც შეგვიძლია Git პროექტების შენახვა და გაზიარება.
3. Git-ის მიერ კონტროლირებად პროექტს ___ ეწოდება.
4. ახალი Git Repository-ის შესაქმნელად გამოიყენება ბრძანება ___.
5. Repository-ის მიმდინარე მდგომარეობის სანახავად გამოიყენება ___.
6. ფაილს, რომელსაც Git ხედავს, მაგრამ ჯერ არ აკონტროლებს, ___ file ეწოდება.

პასუხები

1. Git
2. GitHub

3. Repository
4. `git init`
5. `git status`
6. Untracked

15. სავარჯიშო 2 — Git თუ GitHub?

მოსწავლემ თითოეულთან დაწეროს **Git** ან **GitHub**.

1. ვერსიების კონტროლის სისტემაა.
2. პროექტების ონლაინ შენახვა შეგვიძლია.
3. ლოკალურად, ჩვენს კომპიუტერზე მუშაობს.
4. პროექტის ცვლილებების ისტორიის მართვაში გამოიყენება.
5. პროექტის სხვა ადამიანებისთვის გაზიარება შეგვიძლია.

პასუხები:

1. Git
2. GitHub
3. Git
4. Git
5. GitHub

16. სავარჯიშო 3 — პრაქტიკული

მოსწავლემ დამოუკიდებლად უნდა:

1. შექმნას საქალაქი `my-school-project` ;
2. მასში შექმნას `index.html` ;
3. დაამატოს `<h1>` და `<p>` ;
4. გახსნას Terminal ამ საქალაქში;
5. შეამოწმოს Git:

```
git --version
```

6. შექმნას Git Repository:

```
git init
```

7. შეამოწმოს მდგომარეობა:

```
git status
```

8. იპოვოს `index.html` და განსაზღვროს, რატომ წერია მასთან **Untracked**.

17. სავარჯიშო 4 — იპოვე სწორი ბრძანება

ა) მინდა გავიგო, დაყენებულია თუ არა Git.

```
?
```

ბ) მინდა ჩვეულებრივი საქალაქე Git Repository-ად ვაქციო.

```
?
```

გ) მინდა ვნახო ჩემი Repository-ის მიმდინარე მდგომარეობა.

```
?
```

პასუხები

```
git --version  
git init  
git status
```